TP GUIDÉ — De Cassandra à Spark : traitement des données Vélib

M2 Big Data & IA — Données distribuées · Séance 4

Durée indicative : 1h30 · **Travail individuel**

Technologies : Docker · Cassandra · Jupyter · PySpark · Spark UI

1. Objectif du TP

Dans le TP précédent, vous avez construit un cluster Cassandra de 3 nœuds et stocké des données Vélib' dans :

Keyspace : velib_cluster
Table : stations_velib
La table contient notamment :

station_id
nom_station
capacite
vatiques_disponibles
bornes_disponibles
derniere_mise_a_jour

Le TP3 Cassandra vous a permis de travailler sur le cluster, le **RF = 3**, les niveaux de cohérence `ONE`, `QUORUM`, `ALL` et les pannes de nœuds. Aujourd'hui, nous allons utiliser **Spark pour traiter les données déjà stockées dans Cassandra**.

Objectifs

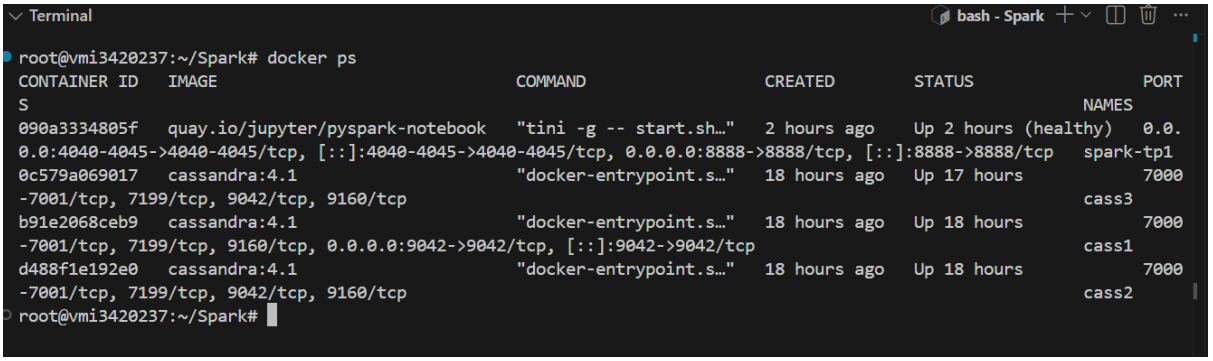
À la fin du TP, vous devez savoir :

- lire des données Cassandra depuis un notebook PySpark ;
- créer un DataFrame Spark ;
- réaliser des traitements métier ;
- comprendre les partitions ;
- observer Jobs, Stages et Tasks ;
- identifier un Shuffle ;
- utiliser la Spark UI.

PARTIE A — Préparer l'environnement

2. Vérifier Cassandra et Spark

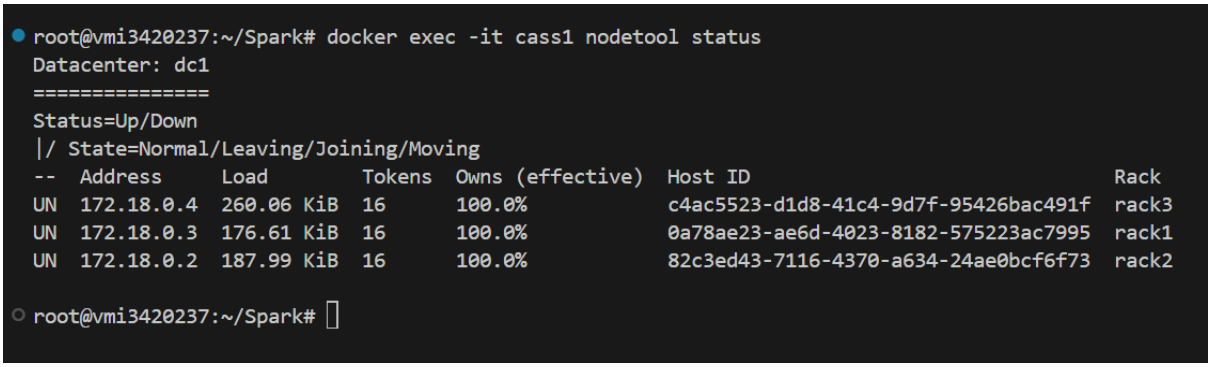
Dans votre terminal hôte, vérifiez que tous les conteneurs tournent correctement :



Vous devez retrouver notamment `cass1`, `cass2`, `cass3` et `spark-tp1`

Vérifiez ensuite l'état du cluster Cassandra :

docker exec -it cass1 nodetool status

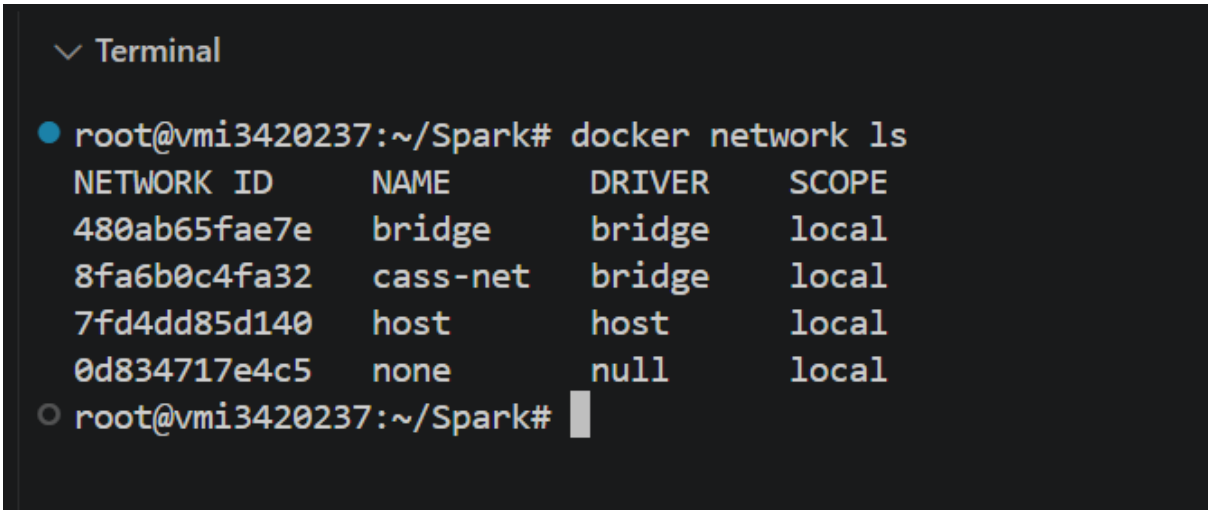


Les trois nœuds doivent indiquer l'état **UN** (*Up/Normal*).

3. Vérifier le réseau Docker

Pour que Spark puisse interroger Cassandra, ils doivent appartenir au même réseau Docker (cass-net).

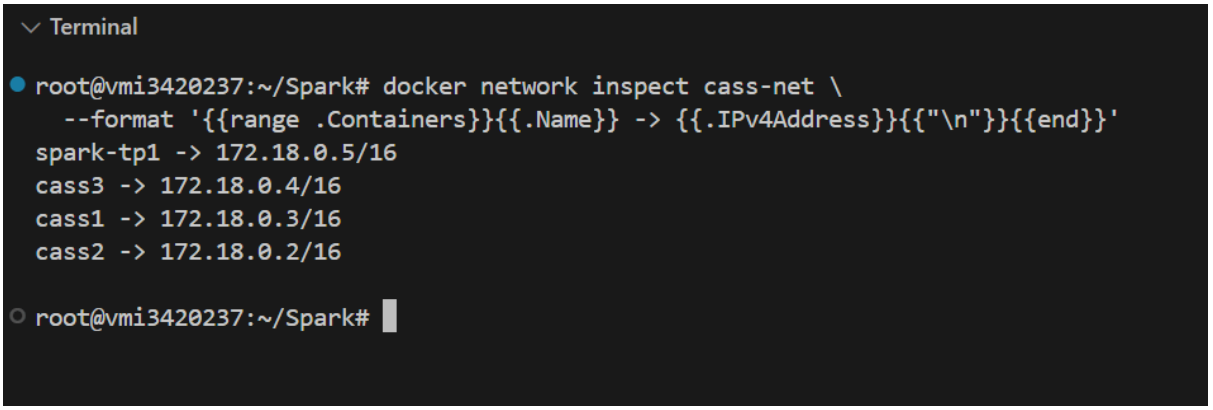
Listez les réseaux disponibles : docker network ls



Connecter Spark à `cass-net` : docker network connect cass-net spark-tp1
Si aucune erreur ne s'affiche, c'est bon.

Vérifie simplement :

```
docker network inspect cass-net \
  --format '{{range .Containers}}{{.Name}} -> {{.IPv4Address}}{{"\n"}}{{end}}'
```



Spark et Cassandra dialoguent directement à travers ce pont réseau `cass-net`.

PARTIE B — Préparer Jupyter et Spark

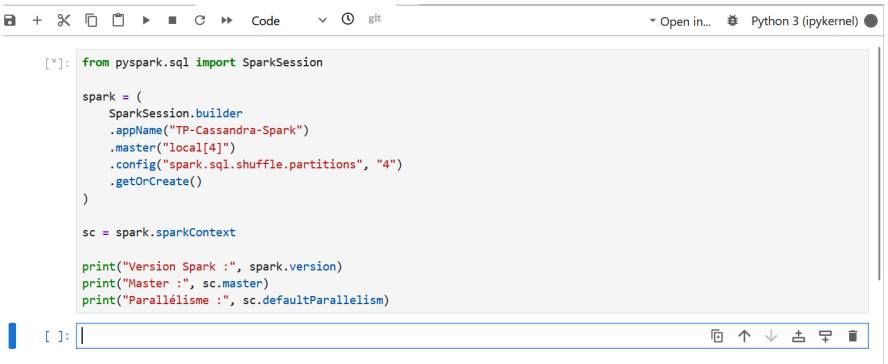
4. Ouvrir Jupyter

Ouvrez votre navigateur sur l'interface de travail :

http://localhost:8888/?token=spark
Dans le dossier `work`, créez un nouveau notebook nommé `tp_cassandra_spark.ipynb`

5. Créer la SparkSession

Dans la première cellule de votre notebook, initialisez le point d'entrée de Spark :



Le paramètre `local[4]` indique ici que Spark utilise 4 threads locaux pour exécuter vos traitements en parallèle sur votre machine.

PARTIE C — Se connecter à Cassandra

6. Installer le driver Cassandra pour Python

Pour connecter Python à Cassandra, nous avons besoin du package officiel `cassandra-driver`. Exécutez directement dans le notebook :

%pip install cassandra-driver

Importez ensuite le module de connexion : from cassandra.cluster import Cluster

7. Tester la connexion

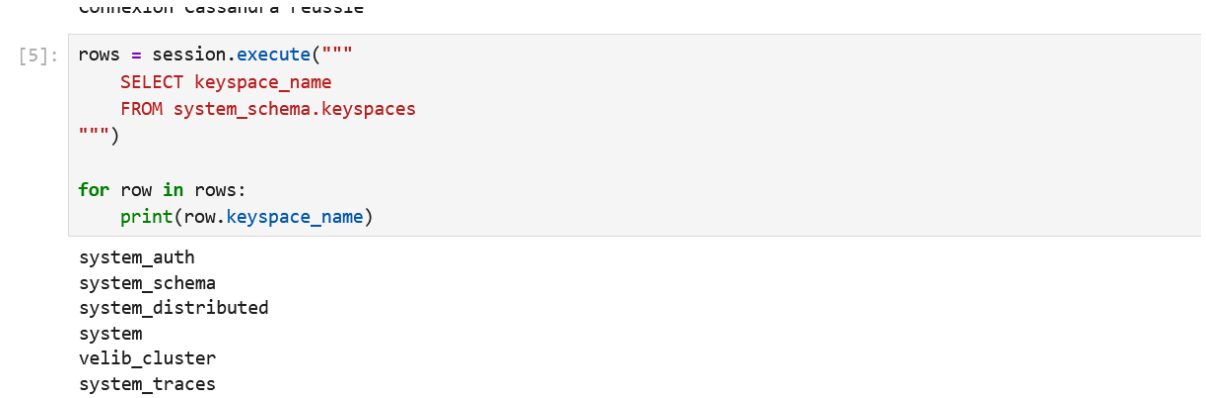
Établissez une session de test vers le premier nœud du cluster (`cass1`) :

cluster = Cluster(["cass1"])

session = cluster.connect()

print("Connexion Cassandra réussie")

Vérifiez que le keyspace de vos données Vélib' est bien présent :



Vous devez voir apparaître `velib_cluster`

PARTIE D — Découvrir les données

8. Sélectionner le keyspace et la table

Pointez sur le keyspace et vérifiez la structure exacte de la table des stations :

[6]:
```
KEYSPACE = "velib_cluster"
TABLE = "stations_velib"

session.set_keyspace(KEYSPACE)

rows = session.execute("""
    SELECT column_name, type
    FROM system_schema.columns
    WHERE keyspace_name = 'velib_cluster'
      AND table_name = 'stations_velib'
""")

for row in rows:
    print(row.column_name, "->", row.type)
```

```
bornes_disponibles -> int
capacite -> int
derniere_mise_a_jour -> timestamp
nom_station -> text
station_id -> text
vatiques_disponibles -> int
```

9. Lire quelques données de validation

Vérifiez un premier extrait de lignes directement depuis Cassandra :

[7]:
```
rows = session.execute("""
    SELECT *
    FROM stations_velib
    LIMIT 10
""")

for row in rows:
    print(row)
```

```
Row(station_id='15047', bornes_disponibles=45, capacite=52, derniere_mise_a_jour=datetime.datetime(2026, 10, 6, 18, 25, 5), nom_station='Morillons - Dantzig', vatiques_disponibles=6)
Row(station_id='9020', bornes_disponibles=15, capacite=21, derniere_mise_a_jour=datetime.datetime(2026, 10, 6, 18, 27, 9), nom_station='Toudouze - Clauzel', vatiques_disponibles=6)
Row(station_id='25006', bornes_disponibles=13, capacite=22, derniere_mise_a_jour=datetime.datetime(2026, 10, 6, 18, 27, 29), nom_station='Place Nelson Mandela', vatiques_disponibles=8)
Row(station_id='32017', bornes_disponibles=2, capacite=22, derniere_mise_a_jour=datetime.datetime(2026, 10, 6, 18, 24, 39), nom_station='Basilique', vatiques_disponibles=15)
Row(station_id='7003', bornes_disponibles=56, capacite=60, derniere_mise_a_jour=datetime.datetime(2026, 10, 6, 18, 24, 38), nom_station='Square Boucicaut', vatiques_disponibles=3)
Row(station_id='14111', bornes_disponibles=24, capacite=25, derniere_mise_a_jour=datetime.datetime(2026, 10, 6, 18, 26, 39), nom_station='Cassini - Denfert-Rochereau', vatiques_disponibles=1)
```

PARTIE E — De Cassandra à un Spark DataFrame

10. Récupérer les données et créer le DataFrame

Récupérons l'ensemble des lignes pour les injecter dans un DataFrame Spark :

```
[8]: rows = session.execute("""
         SELECT *
         FROM stations_velib
     """)

     columns = rows.column_names
     data = [tuple(row) for row in rows]

     print("Colonnes :", columns)
     print("Nombre de lignes récupérées :", len(data))
```

```
Colonnes : ['station_id', 'bornes_disponibles', 'capacite', 'derniere_mise_a_jour', 'nom_station', 'vatiques_disponibles']
Nombre de lignes récupérées : 21
```

```
[ ]:
```

Créez le DataFrame Spark à partir de ces données :

```
[9]: df = spark.createDataFrame(data, columns)

     df.show(10, truncate=False)
     df.printSchema()
     print("Nombre de lignes dans le DataFrame :", df.count())
```

```
+----------+-----------------+--------+---------------------+--------------------------+--------------------+
|station_id|bornes_disponibles|capacite|derniere_mise_a_jour   |nom_station               |vatiques_disponibles|
+----------+-----------------+--------+---------------------+--------------------------+--------------------+
|15047     |45               |52      |2026-10-06 18:25:05    |Morillons - Dantzig       |6                   |
|9020      |15               |21      |2026-10-06 18:27:09    |Toudouze - Clauzel        |6                   |
|25006     |13               |22      |2026-10-06 18:27:29    |Place Nelson Mandela      |8                   |
|32017     |2                |22      |2026-10-06 18:24:39    |Basilique                 |15                  |
|7003      |56               |60      |2026-10-06 18:24:38    |Square Boucicaut          |3                   |
|14111     |24               |25      |2026-10-06 18:26:39    |Cassini - Denfert-Rochereau|1                   |
|8026      |9                |12      |2026-10-06 18:27:17    |Messine - Place Du Pérou  |1                   |
|99999     |15               |30      |2026-10-06 19:53:07.069|Station Test Panne        |15                  |
|5016      |3                |17      |2026-10-06 18:27:25    |Thouin - Cardinal Lemoine |12                  |
|33006     |19               |31      |2026-10-06 18:24:48    |André Karman - République |11                  |
+----------+-----------------+--------+---------------------+--------------------------+--------------------+
only showing top 10 rows
root
 |-- station id: string (nullable = true)
```

PARTIE F — Premiers traitements Spark

11. Sélection et Filtrage

Importez les fonctions utilitaires de Spark :

```
from pyspark.sql import functions as F
```

Sélectionnez les colonnes clés :

[10]:
```
df.select(
    "station_id",
    "nom_station",
    "capacite",
    "vatiques_disponibles",
    "bornes_disponibles"
).show(10, truncate=False)
```

```
+----------+--------------------------+--------+--------------------+------------------+
|station_id|nom_station               |capacite|vatiques_disponibles|bornes_disponibles|
+----------+--------------------------+--------+--------------------+------------------+
|15047     |Morillons - Dantzig       |52      |6                   |45                |
|9020      |Toudouze - Clauzel        |21      |6                   |15                |
|25006     |Place Nelson Mandela      |22      |8                   |13                |
|32017     |Basilique                 |22      |15                  |2                 |
|7003      |Square Boucicaut          |60      |3                   |56                |
|14111     |Cassini - Denfert-Rochereau|25     |1                   |24                |
|8026      |Messine - Place Du Pérou  |12      |1                   |9                 |
|99999     |Station Test Panne        |30      |15                  |15                |
|5016      |Thouin - Cardinal Lemoine |17      |12                  |3                 |
|33006     |André Karman - République |31      |11                  |19                |
+----------+--------------------------+--------+--------------------+------------------+
only showing top 10 rows
```

Appliquez des filtres métiers (par exemple, stations presque vides ou très remplies) :

```
# Stations avec moins de 5 vélos disponibles
df.filter(F.col("vatiques_disponibles") < 5).show(10, truncate=False)

# Stations avec plus de 20 vélos disponibles
df.filter(F.col("vatiques_disponibles") > 20).show(10, truncate=False)
```

[12]:
```
from pyspark.sql import functions as F
```

[13]:
```
# Stations avec moins de 5 vélos disponibles
df.filter(F.col("vatiques_disponibles") < 5).show(10, truncate=False)

# Stations avec plus de 20 vélos disponibles
df.filter(F.col("vatiques_disponibles") > 20).show(10, truncate=False)
```

```
+----------+------------------+--------+-------------------+-----------------------------------
+--------------------+
|station_id|bornes_disponibles|capacite|derniere_mise_a_jour|nom_station                        |vatiques_disponib
les|
+----------+------------------+--------+-------------------+-----------------------------------
+--------------------+
|7003      |56                |60      |2026-10-06 18:24:38 |Square Boucicaut                   |3
|
|14111     |24                |25      |2026-10-06 18:26:39 |Cassini - Denfert-Rochereau        |1
|
|8026      |9                 |12      |2026-10-06 18:27:17 |Messine - Place Du Pérou           |1
|
|32304     |23                |28      |2026-10-06 18:25:34 |Charcot - Benfleet                 |4
|
|42208     |32                |35      |2026-10-06 18:26:13 |Place des Marseillais              |2
|
```

PARTIE G — Transformation métier

12. Calculer le taux de disponibilité

Ajoutons une colonne calculée représentant le taux de remplissage en pourcentage :

[14]:
```
df_taux = df.withColumn(
    "taux_disponibilite",
    F.when(
        F.col("capacite") > 0,
        (F.col("vatiques_disponibles") / F.col("capacite")) * 100
    ).otherwise(None)
)

df_taux.select(
    "station_id",
    "nom_station",
    "capacite",
    "vatiques_disponibles",
    "taux_disponibilite"
).show(10, truncate=False)
```

```
+----------+--------------------------+--------+--------------------+------------------+
|station_id|nom_station               |capacite|vatiques_disponibles|taux_disponibilite|
+----------+--------------------------+--------+--------------------+------------------+
|15047     |Morillons - Dantzig       |52      |6                   |11.538461538461538|
|9020      |Toudouze - Clauzel        |21      |6                   |28.57142857142857 |
|25006     |Place Nelson Mandela      |22      |8                   |36.36363636363637 |
|32017     |Basilique                 |22      |15                  |68.18181818181817 |
|7003      |Square Boucicaut          |60      |3                   |5.0               |
|14111     |Cassini - Denfert-Rochereau|25     |1                   |4.0               |
```

PARTIE H — Agrégations et Catégorisation

13. Statistiques et regroupements

Calculons des métriques globales sur la flotte :

[15]:
```
df.select(
    F.min("vatiques_disponibles").alias("minimum"),
    F.max("vatiques_disponibles").alias("maximum"),
    F.avg("vatiques_disponibles").alias("moyenne")
).show()
```

```
+-------+-------+-----------------+
|minimum|maximum|          moyenne|
+-------+-------+-----------------+
|      1|     23|8.142857142857142|
+-------+-------+-----------------+
```

Catégorisons ensuite les stations selon leur niveau de vélos disponibles (`Faible`, `Moyenne`, `Forte`) :

[16]:
```
df_categorie = df.withColumn(
    "categorie",
    F.when(F.col("vatiques_disponibles") < 5, "Faible")
     .when(F.col("vatiques_disponibles") <= 15, "Moyenne")
     .otherwise("Forte")
)

df_categorie.groupBy("categorie").count().orderBy(F.desc("count")).show()
```

```
+---------+-----+
|categorie|count|
+---------+-----+
|  Moyenne|   13|
|   Faible|    6|
|    Forte|    2|
+---------+-----+
```

PARTIE I — Partitions et Lazy Evaluation

14. Gestion des partitions

Vérifiez le découpage physique actuel de votre DataFrame :

```
print("Nombre de partitions initiales :", df.rdd.getNumPartitions())

# Réorganisation explicite en 4 partitions
df4 = df.repartition(4)
print("Nombre de partitions après repartition(4) :", df4.rdd.getNumPartitions())
```

[17]:
```
print("Nombre de partitions initiales :", df.rdd.getNumPartitions())

# Réorganisation explicite en 4 partitions
df4 = df.repartition(4)
print("Nombre de partitions après repartition(4) :", df4.rdd.getNumPartitions())
```

```
Nombre de partitions initiales : 4
Nombre de partitions après repartition(4) : 4
```

[ ]:

5. Démonstration de l'évaluation paresseuse (*Lazy Evaluation*)

Enchaînons des transformations sans déclencher de calcul immédiat :

[18]:
```
result = (
    df
    .filter(F.col("vatiques_disponibles") > 5)
    .select("station_id", "nom_station", "vatiques_disponibles")
    .orderBy(F.desc("vatiques_disponibles"))
)

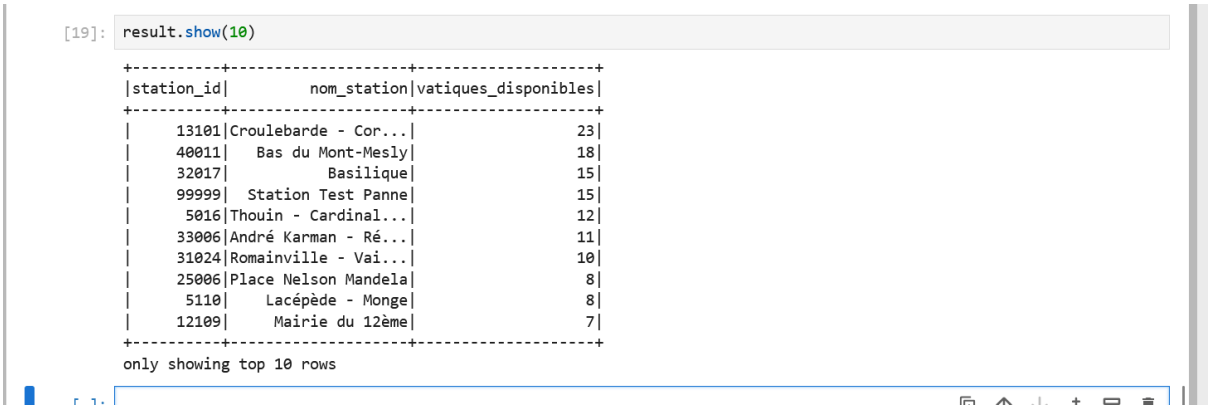
print("Le plan logique est construit, mais aucun calcul n'a été exécuté.")
```

```
Le plan logique est construit, mais aucun calcul n'a été exécuté.
```

[ ]:

Déclenchez l'action pour lancer l'exécution du DAG :

result.show(10)



```
[19]: result.show(10)

+----------+--------------------+--------------------+
|station_id|        nom_station|vatiques_disponibles|
+----------+--------------------+--------------------+
|     13101|Croulebarde - Cor...|                  23|
|     40011|   Bas du Mont-Mesly|                  18|
|     32017|           Basilique|                  15|
|     99999|  Station Test Panne|                  15|
|      5016|Thouin - Cardinal...|                  12|
|     33006|André Karman - Ré...|                  11|
|     31024|Romainville - Vai...|                  10|
|     25006|Place Nelson Mandela|                   8|
|      5110|    Lacépède - Monge|                   8|
|     12109|     Mairie du 12ème|                   7|
+----------+--------------------+--------------------+
only showing top 10 rows
```

PARTIE J — Supervision avec la Spark UI

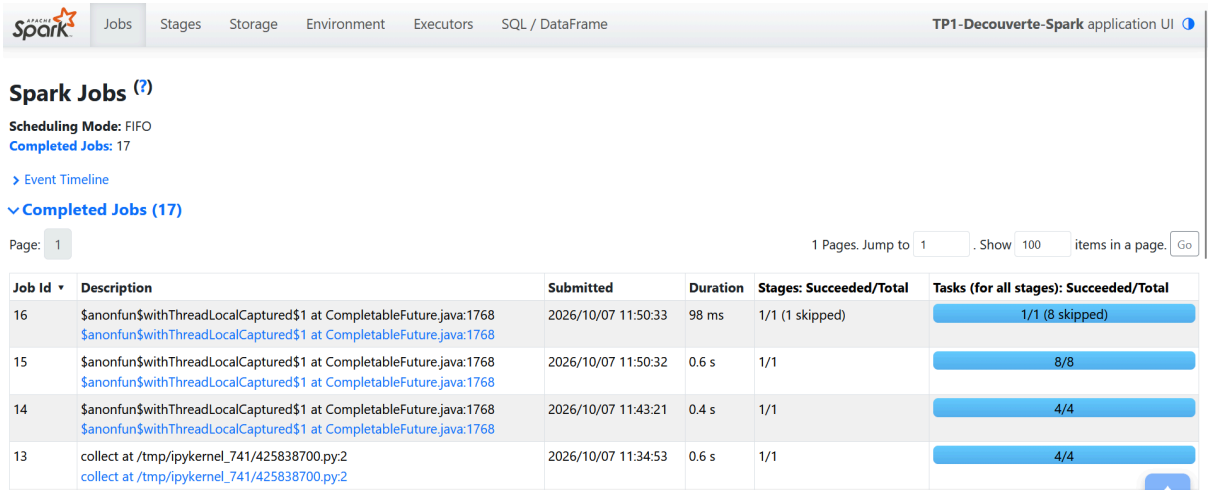
16. Lecture des Jobs, Stages et Tasks

Pendant que votre notebook s'exécute, ouvrez l'interface de monitoring dans votre navigateur :

http://localhost:4040

Relancez l'action result.show(10) et observez la hiérarchie dans l'onglet **Jobs** :

- **Job** (déclenché par l'action)
- **Stages** (coupés par les frontières réseau / shuffle)
- **Tasks** (exécutées en parallèle sur chaque partition)



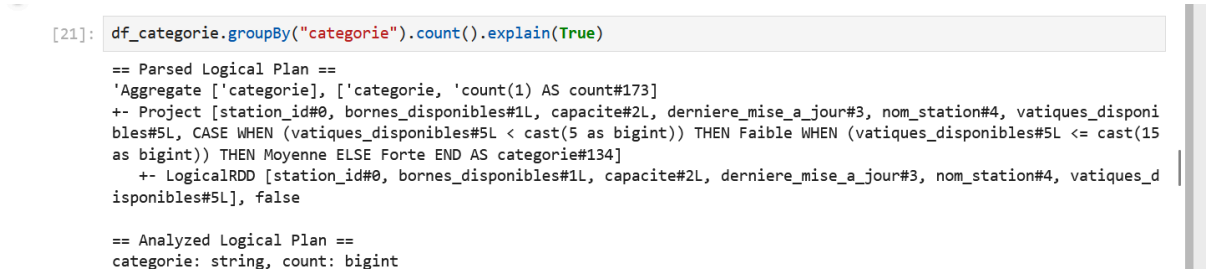
PARTIE K — Comprendre le Shuffle

17. Identifier un Shuffle dans un plan d'exécution

Les opérations impliquant un regroupement ou un tri (comme groupBy(), orderBy(), ou les jointures) forcent Spark à redistribuer les données à travers les nœuds ou les partitions : c'est le **Shuffle**.

Analysez le plan physique d'une requête de regroupement :

df_categorie.groupBy("categorie").count().explain(True)



```
[21]: df_categorie.groupBy("categorie").count().explain(True)

== Parsed Logical Plan ==
'Aggregate ['categorie], ['categorie, 'count(1) AS count#173]
+- Project [station_id#0, bornes_disponibles#1L, capacite#2L, derniere_mise_a_jour#3, nom_station#4, vatiques_disponi
bles#5L, CASE WHEN (vatiques_disponibles#5L < cast(5 as bigint)) THEN Faible WHEN (vatiques_disponibles#5L <= cast(15
as bigint)) THEN Moyenne ELSE Forte END AS categorie#134]
   +- LogicalRDD [station_id#0, bornes_disponibles#1L, capacite#2L, derniere_mise_a_jour#3, nom_station#4, vatiques_d
isponibles#5L], false

== Analyzed Logical Plan ==
categorie: string, count: bigint
```

Recherchez la mention Exchange dans le plan : elle indique l'opération de shuffle. Vous retrouverez ces métriques de transferts dans la Spark UI sous les libellés *Shuffle Read* / *Shuffle Write*.